

Análise do Projeto E-Ponto

Sistema de Controle de Ponto Digital
Requisitos REP-P e Vulnerabilidades de Software

Disciplina: Programação Avançada para Web
Instituição: Universidade Vila Velha (UVV)
Participantes: Lucas André, Roberto Dias, Rychard Chaves
Data: Maio de 2026

Sumário

PARTE 1 — Análise Geral dos Problemas Legais e de Software

- 1.1 Problemas Legais (Legislação Brasileira)
- 1.2 Problemas de Software / Arquitetura
- 1.3 Sugestões de Priorização

PARTE 2 — Requisitos Legais REP-P (Portaria MTP 671/2021)

- 2.1 Definição e enquadramento
- 2.2 Identificação obrigatória em cada registro
- 2.3 NSR — Número Sequencial de Registro
- 2.4 Comprovante de Registro de Ponto
- 2.5 AFD — Arquivo Fonte de Dados
- 2.6 AEJ — Arquivo Eletrônico de Jornada
- 2.7 O que o sistema NÃO PODE fazer
- 2.8 Assinatura digital e integridade
- 2.9 Retenção e auditoria
- 2.10 Tolerâncias e regras CLT integradas

PARTE 3 — Vulnerabilidades e Cuidados de Software

- 3.1 Manipulação de tempo
- 3.2 Geolocalização
- 3.3 Autenticação e sessão
- 3.4 Controle de acesso (autorização)
- 3.5 OWASP Top 10 aplicado ao Flask
- 3.6 Vulnerabilidades específicas de Flask
- 3.7 Banco de dados
- 3.8 Concorrência e race conditions
- 3.9 LGPD na prática técnica
- 3.10 Disponibilidade
- 3.11 Exportação de relatórios
- 3.12 Frontend

PARTE 4 — Checklist Prático antes de Codar

Parte 1 — Análise Geral

1.1 Problemas Legais (Legislação Brasileira)

1.1.1 Portaria MTP nº 671/2021

Esta é a norma central que NÃO é citada no documento original do projeto e define rigorosamente o que pode ou não ser chamado de "sistema de ponto" no Brasil. Existem três modalidades:

Modalidade	O que é	Exigências
REP-C	Equipamento físico	Hardware certificado pelo INMETRO
REP-A	Sistema alternativo	Exige acordo ou convenção coletiva com o sindicato
REP-P	Programa (software)	Não exige acordo coletivo, mas tem requisitos técnicos rígidos

Problema crítico:

O E-Ponto, da forma como está descrito, seria um REP-P, mas o documento ignora todos os requisitos obrigatórios desse formato: AFD, AEJ, comprovante de batida, assinatura digital, identificação obrigatória e proibição de restrições à marcação.

1.1.2 CLT — Art. 74

- Obrigatório apenas para empresas com mais de 20 funcionários. O público-alvo de "pequenas e médias empresas" precisa ser refinado.
- Banco de horas só é válido com acordo individual escrito (até 6 meses), acordo coletivo (até 1 ano) ou convenção coletiva.

1.1.3 LGPD — Lei 13.709/2018

Ponto mais negligenciado do documento original. Problemas:

1. Geolocalização: dado pessoal sensível por contexto. Precisa de base legal explícita (execução de contrato + legítimo interesse, nunca consentimento puro pela subordinação). Jurisprudência do TST veda geolocalização contínua — só no momento da batida.
2. Foto opcional: se usada para reconhecimento facial, vira dado biométrico sensível (art. 11 LGPD).
3. Falta no documento: DPO/encarregado, política de retenção (5 anos), direitos do titular, ROPA, plano de resposta a incidentes.

1.1.4 Outros pontos legais ignorados

- Intervalo intrajornada: alerta quando inferior a 1h (jornadas > 6h) — gera horas extras automáticas.
- Intervalo interjornada: 11h mínimas entre jornadas.
- Trabalho noturno: hora reduzida (52min30s) e adicional de 20%.
- DSR: faltas injustificadas afetam o cálculo do descanso semanal remunerado.

1.2 Problemas de Software / Arquitetura

1.2.1 Integridade e auditoria dos registros

"Aprovação ou ajuste manual de registros" sem trilha de auditoria imutável é ilegal e tecnicamente perigoso. Precisa de log append-only, hash encadeado para detectar adulteração e versionamento (registro original nunca apagado).

1.2.2 Sincronização de horário

Não há menção a NTP autoritativo. O horário do cliente (browser) não pode ser fonte da verdade — funcionário pode adulterar. Servidor precisa carimbar tempo no backend, sincronizar com NTP confiável e armazenar timezone explícito.

1.2.3 Geolocalização — problemas técnicos

- Spoofing trivial via DevTools → Sensors → Override location.
- GPS de celular com precisão de 5-50m em ambientes fechados.
- IP geolocation é ainda pior (precisão de cidade).
- Sem prova criptográfica de localização, não tem valor legal.

1.2.4 Autenticação fraca

"Login com e-mail e senha" é o mínimo e insuficiente: sem 2FA para admin, sem rate limiting, sem política de senha, sem hash adequado mencionado, sem detecção de sessão suspeita.

1.2.5 Stack incompatível com o projeto

Escolha	Problema
SQLite dev / Postgres prod	Diferenças de comportamento causam bugs (tipos de data, constraints, locking)
Pandas para exportar Excel	Overkill — para 6000 registros, openpyxl puro é mais leve e rápido
Flask puro	Sem workers (Gunicorn), sem fila (Celery), sem cache (Redis)
Sem ORM migrations	Não mencionam Alembic — em produção é um pesadelo

1.2.6 Escalabilidade declarada vs realidade

"200 funcionários" é limite baixo. Em pico (8h da manhã), são 200 requisições simultâneas — Flask single-threaded não aguenta. Não esclarece arquitetura multi-tenant (por schema, tabela ou instância).

1.2.7 Funcionalidades faltantes

- Modo offline para "trabalho externo eventual" — sem PWA com cache local e sync, isso quebra.
- Backup e disaster recovery — registros têm valor probatório judicial.
- eSocial — obrigatório para empresas com empregados (eventos S-2230, S-1200).
- Modelo de dados de escalas: 12x36, 6x1, jornada flexível, feriados, afastamentos.

1.3 Sugestões de Priorização

Antes do MVP, vocês precisam decidir:

1. Escopo legal: conformidade com Portaria 671 (REP-P) ou "controle interno informal"? Isso muda tudo.
2. Multi-tenant: definir arquitetura antes de escrever a primeira linha.
3. Modelo de dados de jornada: as regras da CLT são complexas; modelem isso primeiro.
4. LGPD by design: DPIA, política de privacidade, controle de retenção desde o dia 1.

Parte 2 — Requisitos REP-P (Portaria 671/2021)

2.1 Definição e enquadramento

REP-P = Registrador Eletrônico de Ponto via Programa. É software (sem hardware dedicado), pode rodar em servidor, desktop, web ou mobile. NÃO exige certificação INMETRO (diferente do REP-C) nem acordo coletivo (diferente do REP-A), MAS tem requisitos técnicos rigorosos.

Base legal: Portaria MTP nº 671/2021, arts. 80 a 91 (capítulo do REP-P) e Anexos I, II e III.

2.2 Identificação obrigatória em cada registro (Art. 84)

Campo	Detalhe
CNPJ/CPF do empregador	Identificação fiscal completa
CEI/CAEPF (se aplicável)	Cadastro específico
Local da prestação do serviço	Endereço onde o trabalho ocorre
Identificação do empregado	Nome + PIS/NIS ou CPF
Data e horário	Com precisão de segundos
Tipo de operação	Entrada / saída / inclusão / alteração
NSR	Número Sequencial de Registro (único, contínuo, jamais reutilizado)
Hash do registro	Para garantir integridade

2.3 NSR — Número Sequencial de Registro

Conceito fundamental e frequentemente mal implementado:

- Sequencial estritamente crescente dentro do empregador.
- Nunca pode ter gap (sem números pulados).
- Nunca pode ser reutilizado (mesmo se um registro for "estornado").
- Atribuído pelo servidor no momento da gravação, nunca pelo cliente.
- Implementação: use uma sequence do PostgreSQL com lock, NÃO MAX(nsr)+1 (race condition).

```
# ERRADO (race condition)
ultimo = db.session.query(func.max(Registro.nsr)).scalar()
novo = Registro(nsr=ultimo+1)

# CERTO (sequence atomica)
# No Postgres: CREATE SEQUENCE nsr_empregador_X;
# db.session.execute(text("SELECT nextval('nsr_empregador_X')"))
```

2.4 Comprovante de Registro de Ponto (Art. 84, §1º)

Toda batida deve gerar comprovante entregue ao funcionário, contendo:

- Cabeçalho com identificação do empregador e empregado.
- NSR.
- Data/hora da batida.
- Tipo de marcação.

- Hash de identificação (SHA-256 do registro).

Forma de entrega aceitável: PDF/A baixável pelo funcionário a qualquer momento, ou e-mail automático (cuidado com dependência de SMTP externo). O funcionário precisa poder baixar comprovantes antigos por 5 anos.

2.5 AFD — Arquivo Fonte de Dados (Anexo I)

Layout textual fixo, uma linha por evento, codificação ASCII puro (não UTF-8), com tipos de registro:

Tipo	Conteúdo
1	Cabeçalho do arquivo (empregador, período)
2	Identificação da empresa no REP
3	Marcação de ponto
4	Ajuste de relógio (mudança de hora)
5	Inclusão/alteração/exclusão de empregado
6	Eventos sensíveis (manutenção, falha)
7	Empregador (CNPJ, razão social)
9	Trailer (totalizadores)

Precisa ser gerado sob demanda quando solicitado pelo Auditor Fiscal do Trabalho ou pelo próprio empregador. Deve cobrir qualquer intervalo de datas dos últimos 5 anos.

2.6 AEJ — Arquivo Eletrônico de Jornada (Anexo II)

Diferente do AFD: contém a jornada APURADA (não só batidas brutas): horas trabalhadas, horas extras, atrasos, faltas, banco de horas, justificativas e abonos. Também em formato texto fixo.

2.7 O que o sistema NÃO PODE fazer (Art. 85)

Pontos onde projetos universitários costumam errar:

1. Restringir a marcação — não pode bloquear o funcionário de bater ponto fora do horário previsto. Tudo é registrado; o RH analisa depois.
2. Marcar ponto automático — sem batida do funcionário, não há registro.
3. Alterar marcações já feitas — ajustes só por registro adicional identificado como retificação.
4. Exigir autorização para marcar — funcionário tem direito de bater ponto sempre.
5. Arredondar horários — registre o segundo exato; arredondamentos só para cálculo e simétricos (até 5min, mesmo critério entrada e saída).
6. Apagar registros — deletar é proibido; só "anular" via nova marcação.

2.8 Assinatura digital e integridade

A Portaria exige garantia de inviolabilidade:

- Cada registro deve ter hash criptográfico.
- O AFD precisa estar assinado digitalmente (certificado ICP-Brasil A1 ou A3, ou outro mecanismo aceito).
- Implementação prática: hash encadeado (cada registro inclui hash do anterior — efeito

blockchain), assinatura do bloco diário ou semanal.

```
registro.hash = SHA256(  
    nsr || cnpj || pis || timestamp || tipo || registro_anterior.hash  
)
```

2.9 Retenção e auditoria

- 5 anos mínimo de retenção de todos os dados (Art. 78).
- Logs de acesso administrativo: quem alterou, quando, o quê, justificativa.
- Auditor Fiscal pode pedir os arquivos a qualquer momento.

2.10 Tolerâncias e regras CLT integradas

Regra	Valor
Tolerância na marcação	5 min por batida, máximo 10 min/dia (Art. 58 §1º CLT)
Intervalo intrajornada	1h para jornadas > 6h; 15min para 4h-6h
Intervalo interjornada	11h consecutivas
Hora noturna	22h-5h, reduzida (52min30s), adicional 20%
DSR	1 dia/semana, preferencialmente domingo
Banco de horas	Compensar em até 6 meses (acordo individual) ou 1 ano (coletivo)

Parte 3 — Vulnerabilidades de Software

3.1 Manipulação de tempo (alto risco)

Vetor de ataque: funcionário ou admin malicioso altera horário de batidas.

Vulnerabilidade	Mitigação
Cliente envia timestamp	NUNCA confie no horário do cliente. Use <code>datetime.now(timezone.utc)</code> no servidor
Servidor com hora errada	NTP autoritativo (<code>pool.ntp.br</code>), monitoramento de drift
Replay de requisição	Token CSRF + nonce único por requisição
Timezone misturado	Sempre armazene em UTC, converta na exibição

3.2 Geolocalização — vetor crítico

A geolocalização no navegador é trivialmente falsificável:

- DevTools → Sensors → Override geolocation.
- Extensões como "Location Guard".
- Mock location em Android (modo desenvolvedor).
- VPN + GPS spoofing apps.

Mitigações (nenhuma é perfeita):

1. Combinar com IP geolocation e comparar.
2. Exigir HTTPS obrigatório (geolocation API só funciona em HTTPS).
3. Salvar precisão reportada (`coords.accuracy`) — rejeitar > 100m.
4. Definir geofence (raio aceitável) por local de trabalho.
5. Flag de "suspeito" em vez de bloqueio (deixar RH decidir).
6. Em mobile app nativo: usar attestation (Google Play Integrity, Apple DeviceCheck).

Importante:

Geolocalização NÃO é obrigatória pela Portaria 671 para REP-P; é um plus seu. Se a precisão atrapalhar funcionário legítimo, pondere remover.

3.3 Autenticação e sessão

Vulnerabilidade	Mitigação
Senhas fracas	Mínimo 12 chars, complexidade, verificar contra <code>haveibeenpwned</code>
Hash inadequado	<code>bcrypt</code> (cost >= 12) ou <code>argon2id</code> — nunca MD5/SHA1/SHA256 puro
Brute force	Rate limiting por IP + usuário (5 tentativas/15min) → captcha →
Credential stuffing	Notificação por e-mail em login de novo dispositivo
Session fixation	Regenerar <code>session_id</code> após login
Session hijacking	Cookies com <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Strict</code>
Sessão eterna	Timeout de inatividade (30min admin, 8h funcionário)
2FA ausente	Obrigatório para admin/RH; opcional para funcionário (TOTP)
Senha em log	Logger filter para nunca registrar password, token, cookie

3.4 Controle de acesso (autorização)

O OWASP A01:2021 — Broken Access Control é o problema #1 de webapps.

```
# ERRADO - IDOR (Insecure Direct Object Reference)
@app.route('/registro/<int:id>')
def ver_registro(id):
    return Registro.query.get(id) # qualquer um ve qualquer registro!

# CERTO
@app.route('/registro/<int:id>')
@login_required
def ver_registro(id):
    reg = Registro.query.get_or_404(id)
    if reg.usuario_id != current_user.id and not current_user.is_admin:
        abort(403)
    return reg
```

Vulnerabilidades específicas:

- IDOR: funcionário acessa /registro/<id> de colega trocando o número.
- Privilege escalation: campo is_admin editável via form (sempre filtrar entrada).
- Multi-tenant leak: empresa A vê dados da empresa B (sempre filtrar por empresa_id em todo query).
- Mass assignment: User(**request.form) permite setar role='admin' — sempre use whitelist explícita.

3.5 OWASP Top 10 aplicado ao Flask

A02:2021 — Cryptographic Failures

- SECRET_KEY no .env, nunca commitado.
- HTTPS obrigatório (Strict-Transport-Security header).
- Senha hashada com bcrypt/argon2.
- Dados pessoais em repouso: Postgres com TDE ou criptografia de coluna para CPF/PIS.

A03:2021 — Injection

- Use sempre ORM com parâmetros (SQLAlchemy faz por padrão).
- NUNCA use f"... WHERE id = {id}" em SQL raw.
- Validação de entrada com WTForms ou Pydantic.
- Template Jinja2 com autoescape ligado (padrão Flask).

A04:2021 — Insecure Design

- Threat modeling antes de codar.
- Funcionário não pode ser admin da própria empresa por acidente (separação de papéis).

A05:2021 — Security Misconfiguration

- DEBUG=False em produção (parece óbvio mas falha sempre).
- Debug Toolbar desativado em produção.
- Headers de segurança: use flask-talisman.
- Mensagens de erro genéricas para usuário, detalhe só no log.

A06:2021 — Vulnerable Components

- pip-audit ou safety check no CI.
- Atualizar dependências (Dependabot).
- Pin de versões no requirements.txt.

A08:2021 — Software & Data Integrity Failures

- Hash dos registros de ponto (Portaria 671 exige).
- Validar uploads (foto): magic bytes, não extensão.

A09:2021 — Security Logging

- Logar: login sucesso/falha, alterações de registro, exportações, mudanças de perfil.
- NÃO logar: senha, token, dados sensíveis em claro.
- Logs em local seguro, retenção 5 anos.

A10:2021 — SSRF

- Se houver integração externa (folha, webhooks), validar URLs alvo (whitelist).

3.6 Vulnerabilidades específicas de Flask

Problema	Solução
CSRF em forms	Flask-WTF + token em todos POST
CSRF em APIs JSON	Tokens via header X-CSRF-Token ou SameSite=Strict
XSS via Jinja2	Nunca use safe em dados de usuário
pickle para sessão	Flask usa itsdangerous (ok), nunca desserializar dados não confiáveis
Open redirect	Validar next parameter contra whitelist
Path traversal em uploads	secure_filename() + verificação de path absoluto
ReDoS (regex)	Cuidado com regex complexa em input de usuário

3.7 Banco de dados

- Migrations versionadas: Alembic (flask db migrate) — nunca db.create_all() em produção.
- Backup automático: dump diário, retenção 90 dias, teste de restore mensal.
- Soft delete: registros de ponto nunca deletados — flag deleted_at ou tabela de histórico.
- Constraints no DB: NOT NULL, UNIQUE, CHECK — não confie só na aplicação.
- Pool de conexões: SQLALCHEMY_ENGINE_OPTIONS = {'pool_pre_ping': True}.
- Read replica para relatórios pesados (se crescer).

3.8 Concorrência e race conditions

1. Geração do NSR: use sequence/locking.
2. Batida dupla: clique 2x rápido → 2 registros → idempotency key + constraint unique (usuario_id, timestamp_truncado_segundo).
3. Cálculo de banco de horas em paralelo com edição: transactions com isolation REPEATABLE READ.

3.9 LGPD na prática técnica

- Pseudonimização: em logs e analytics, usar ID interno em vez de CPF.
- Direito ao esquecimento vs. obrigação legal: registros ficam 5 anos por lei → prevalece a obrigação legal.
- Portabilidade: usuário pode exportar seus dados em formato legível.
- Acesso: usuário pode ver tudo que o sistema tem dele.
- Consentimento: para uso secundário (analytics, marketing) — separado do consentimento de uso do sistema.

- DPIA: relatório de impacto, exigido para tratamento de larga escala.
- Termos de uso + política de privacidade: aceite versionado, com data e IP.

3.10 Disponibilidade

Se o sistema cair às 8h, 200 funcionários não conseguem bater ponto. Isso gera passivo trabalhista para a empresa cliente.

Mitigações:

- SLA documentado (ex: 99,5% mensal = 3h45min de downtime/mês).
- Modo offline na PWA: bate ponto local, sincroniza quando volta.
- Fallback: e-mail com hash da batida quando sistema indisponível.

3.11 Exportação de relatórios

- Excel com fórmulas injetadas (Formula Injection): célula iniciando com =, +, -, @ executa fórmula. Prefixe com aspas simples.
- Geração síncrona trava worker: use fila (Celery + Redis) para relatórios > 5s.
- Permissão: usuário só exporta seus dados; admin só da sua empresa.

3.12 Frontend

- CSP (Content Security Policy) configurada — Bootstrap CDN precisa estar na lista.
- SRI (Subresource Integrity) em CDNs.
- Inputs com autocomplete=off em campos sensíveis.
- Não armazenar token em localStorage (XSS rouba) — preferir cookie HttpOnly.

Parte 4 — Checklist Prático antes de Codar

Antes de escrever a primeira linha, alinhem com o professor e entre os membros do grupo:

- ☐ Vamos gerar AFD/AEJ no formato exato da Portaria 671 (Anexos I e II)?
- ☐ Vamos emitir comprovante de batida em PDF para o funcionário?
- ☐ Como vamos garantir integridade: hash encadeado, assinatura digital, ambos?
- ☐ Multi-tenant: uma instância por empresa ou todas no mesmo banco com empresa_id?
- ☐ 2FA: obrigatório só para admin ou para todos os usuários?
- ☐ Geolocalização: bloqueia a batida ou só registra como informação?
- ☐ Onde será hospedado? (afeta backup, HTTPS, DPO, latência)
- ☐ Quem é o DPO/encarregado de dados na simulação?
- ☐ Vamos usar Alembic para migrations desde o início?
- ☐ Vamos configurar CI com pip-audit e flake8?
- ☐ Política de retenção: como apagar dados após 5 anos automaticamente?
- ☐ Tolerância CLT: 5min/batida e 10min/dia configuráveis?
- ☐ Como tratar fuso horário se a empresa tiver funcionários em estados diferentes?
- ☐ Modelo de jornada: vamos suportar 12x36, 6x1, jornada flexível ou só padrão 8h?
- ☐ Feriados nacionais, estaduais e municipais — fonte de dados?
- ☐ Banco de horas: prazo de compensação configurável (6 meses ou 1 ano)?

Próximos passos sugeridos:

Definir o modelo de dados (SQLAlchemy) considerando NSR, hash encadeado, soft-delete e multi-tenant; em seguida, criar o plano de implementação faseado para os três participantes dividirem as tarefas.